

ASSIGNMENT 01

Experimental Performance Analysis of Quicksort Pivot Selection Strategies

□ **Deadline: 14 July 2026, 11:59 PM**

Learning Objectives

- Implement Quicksort with four pivot selection strategies.
- Measure and compare execution times across array types and sizes.
- Relate empirical results to theoretical complexity (best, average, worst case).

Problem Statement

Implement Quicksort using the four pivot selection strategies described below. Measure only the execution time of the Quicksort function using `std::chrono`. Conduct the experiments across three input types: random, ascending, and descending arrays.

Pivot Selection Strategies

Strategy	Description
First Element	Always picks the first element of the subarray as the pivot. Fast but performs poorly on ascendingly pre-sorted data — degenerates to $O(n^2)$.
Last Element	Always picks the last element. Symmetric risks to first-element strategy; descending pre-sorted data.
Middle Element	Picks the element at index $\lfloor n/2 \rfloor$. Often avoids worst-case on sorted input.
Random Element	Selects a uniformly random element. Provides probabilistic $O(n \log n)$ guarantee regardless of input order.

Experiment 1 — Random Arrays

Array sizes: 10, 20, 30, ..., 5000 (step = 10) [**N.B.:** You can choose the LIMIT (5000) more if you wish]

Procedure

1. Generate a random array of size n .
2. Create four identical copies of that array.
3. Sort each copy using one of the four pivot strategies.
4. Measure **only the Quicksort execution time** (not copying or generation in step 1/2/3).

5. Repeat steps 1–4 a total of **m** times. **[N.B.:** You can choose the **m** (10 at least) more if you wish]
6. Compute the average execution time across all **m** repetitions.
7. Record the average time in the output table.

Expected Output

- Table 1 – Average execution times (rows = array sizes, columns = strategies)
- Graph 1 – Four curves on one plot (First, Last, Middle, Random pivot)

Experiment 2 — Sorted Arrays

Repeat the full procedure from Experiment 1 twice — once for ascendingly sorted arrays and once for descendingly sorted arrays. These inputs expose worst-case behaviour for naïve pivot choices.

Expected Outputs

- Table 2 + Graph 2A – Ascending input, four curves
- Table 3 + Graph 2B – Descending input, four curves

□ Time Measurement

Use `std::chrono` exclusively. Wrap only the Quicksort call — exclude array generation, copying, and I/O. You may report times in nanoseconds, microseconds, or milliseconds; be consistent throughout the report.

Sample Timing Code (C++)

```
//C++ Timing Template
#include <iostream>
#include <chrono>

using namespace std;
using namespace chrono;

void algorithm() {
    // TODO: implement your Quicksort here
}

int main() {
    auto start = high_resolution_clock::now();

    algorithm();    // call your Quicksort

    auto stop = high_resolution_clock::now();

    auto ns = duration_cast<nanoseconds>(stop - start);
    auto us = duration_cast<microseconds>(stop - start);
    auto ms = duration_cast<milliseconds>(stop - start);
    duration<double> s = stop - start;

    cout << "Execution Time:\n";
    cout << "  Nanoseconds   : " << ns.count() << " ns\n";
    cout << "  Microseconds  : " << us.count() << " us\n";
    cout << "  Milliseconds  : " << ms.count() << " ms\n";
    cout << "  Seconds       : " << s.count() << " s\n";
    return 0;
}
```

System Configuration

Record your actual machine specifications below. The values in the table are sample placeholders — update them with your real system details before submitting.

Item	Specification
Brand	Lenovo
CPU	4.9 GHz
Generation	13th
Processor	Core i7
Cores / Threads	10C/16T
RAM	16 GB DDR5
Cache Memory	24 MB
Storage	512 GB SSD
Operating System	Windows 11 pro
Compiler	GNU GCC
IDE	Code::Blocks

Report Contents

Your submitted PDF must include all of the following sections:

- Algorithm Implemented (in cpp)
- Experimental Results (all tables and graphs)
- Theoretical Comparison: best, average & worst-case analysis
- Discussion & Interpretation of results
- Full C++ source code
- System Configuration

Submission

Submit a **single** PDF file containing:

- All C++ source code
- All tables and graphs (Experiment 1 & 2)
- Complexity analysis and discussion
- System configuration

⚠ Plagiarism Penalty: If any two or more reports are found to be similar or copied, each offending report will receive a penalty of –100% of the assignment marks.

Evaluation — 100 Marks

Criteria	Marks
Implementation of Experimental Setup (Full Code)	15
Tables	25
Graphs	35
Analysis	15
Report Quality	10
TOTAL	100

Note: The weight of this assignment in the final grade will be announced later.

Deadline: 14th July 2020 11:59 PM

Good luck! Write clean, well-commented code and let the data tell the story.